

Programming Languages

Procedures and Environments



Topic Objectives:

1. Review the syntactic way of defining and calling functions and procedures.
2. Discuss the procedure semantic.
3. Identify the different parameter-passing mechanisms.



Introduction

Many languages make strong syntactic distinctions between procedures and functions.

- functions should produce a value only and have no side effects.
- while procedures produce no values and operate by producing side effects.

The distinction between procedures and functions is in essence the same as the difference between expressions and statements.

Procedure calls are *statements*, while function calls are *expressions*.

Procedure Definition and Activation

A *procedure* is a mechanism in a programming language for abstracting a group of actions or computations. The group of actions is called the *body* of the procedure, with the body of the procedure represented as a whole by the name of the procedure. A procedure is defined by providing a *specification* or *interface* and a body.

The specification gives the name of the procedure, a list of the types and names of its *formal parameters*, and the type of its returned value, if any:

```
// C++ code
void intswap (int& x, int& y){ // specification
    int t = x;    // body
    x = y;        // body
    y = t;        // body
}
```

Procedure Definition and Activation

In some languages, and in some situations, a procedure specification can be separated from its body, when the specification must be available in advance:

```
void intswap (int&, int&); // specification only
```

Note that this specification does not require the names of the formal parameters to be specified. In C++ this is called a *declaration*, while the complete definition is called a *definition*. (In C, declarations are called *prototypes*.)

You *call*, or *activate*, a procedure by stating its name, together with arguments to the call, which correspond to its formal parameters:

```
intswap (a, b);
```

Procedure Definition and Activation

A call to a procedure transfers control to the beginning of the body of the called procedure (the *callee*).

When execution reaches the end of the body, control is returned to the *caller*.

In some languages, control can be returned to the caller before reaching the end of the callee's body by using a *return-statement*:

```
// C++ code
void intswap (int& x, int& y){
    if (x == y) return;
    int t = x;
    x = y;
    y = t;
}
```

Procedure Definition and Activation

Functions may or may not also change their parameters and nonlocal variables. In C and C++, all procedures are implicitly functions; those that do not return values are declared *void*, while ordinary functions are declared to have the (return) type of the value that they return:

```
int max (int x, int y) {  
    return x > y ? x : y;  
}
```

Procedure Semantics

From the point of view of semantics, a procedure is a block whose declaration is separated from its execution. For example, in C, blocks *A* and *B* in the following code are executed as they are encountered:

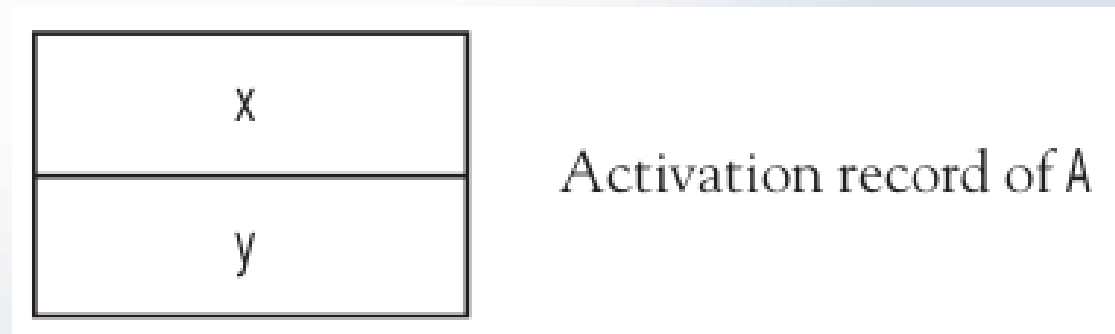
```
A:
{ int x, y;
  ...
  x = y * 10;
  B:
  { int i;
    i = x / 2;
    ...
  } /* end B */
} /* end A */
```

In a block-structured language, when a block is encountered during execution, it causes the allocation of local variables and other objects corresponding to the declarations of the block.

Procedure Semantics

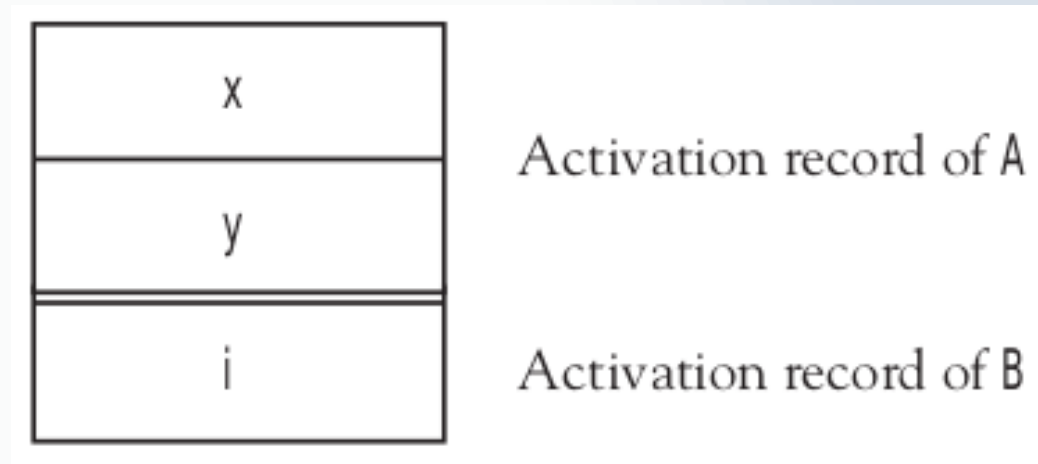
This memory allocated for the local objects of the block is called the *activation record* (or sometimes *stack frame*) of the block. The block is said to be *activated* as it executes under the bindings established by its activation record.

For example, in the preceding C code, during execution *x* and *y* are allocated in the activation record of block *A*.



Procedure Semantics

When block *B* is entered, space is allocated in the activation record of *B* .



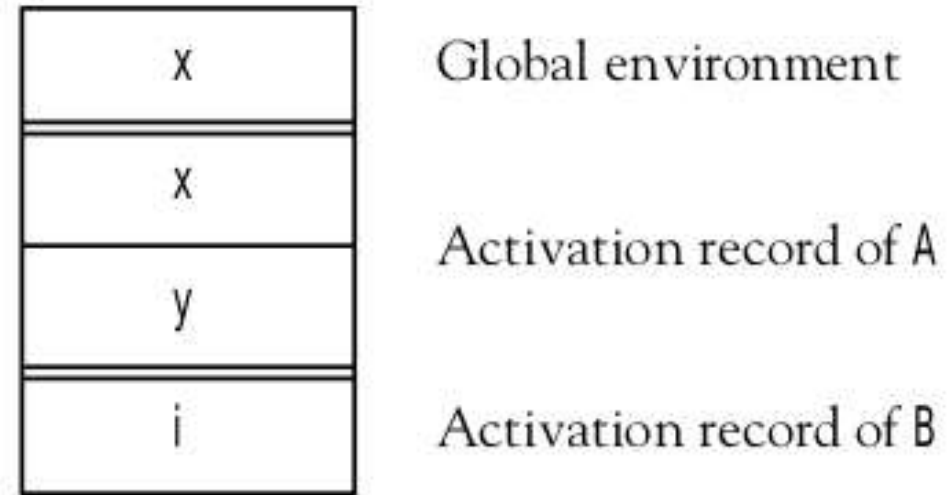
In the preceding code, block *B* needs to access the variable *x* declared in block *A*. A reference to *x* inside *B* is a *nonlocal* reference, since *x* is not allocated in the activation record of *B*, but in the activation record of the surrounding block *A*.

Now consider what would happen if *B* were a procedure called from *A* instead of a block entered directly from *A* . Let's have the following example in C syntax:

Procedure Semantics

```
(1) int x;  
(2) void B(void)  
(3) { int i;  
(4)   i = x / 2;  
(5)   ...  
(6) } /* end B */  
(7) void A(void)  
(8) { int x, y;
```

```
(9)   ...  
(10)  x = y * 10;  
(11)  B();  
(12) } /* end A */  
(13) main()  
(14) { A();  
(15)   return 0;  
(16) }
```



The activation of B must retain information about the global environment, since the nonlocal reference to x will be found there instead of in the activation record of A . This is because the global environment is the *defining environment* of B , while the activation record of A is the *calling environment* of B . Sometimes the defining environment is called the *static environment* and the control environment the *dynamic environment*.

Procedure Semantics

The method of communication of a procedure with its calling environment is through parameters. A parameter list is declared along with the definition of the procedure.

```
int max (int x, int y){  
    return x > y ? x : y;  
}
```

Here, x and y are parameters to the max function. They do not take on any value until max is called, when they are replaced by the arguments from the calling environment, as in:

```
z = max(a+b,10);
```

To emphasize the fact that parameters have no value until they are replaced by arguments, parameters are sometimes called *formal parameters*, while arguments are called *actual parameters*.

Parameter-Passing Mechanisms

Some languages (C, Java, functional languages) offer only one basic kind of parameter-passing mechanism, while others (C++) may offer two or more. In languages with one mechanism, it is also often possible to imitate other mechanisms by using indirection or other language features.

The four important parameter-passing mechanisms: *pass by value*, *pass by reference*, *pass by value-result*, and *pass by name*.

Pass by Value

Pass by value, by far the most common mechanism for parameter passing, the arguments are expressions that are evaluated at the time of the call, with the arguments' values becoming the values of the parameters during the execution of the procedure.

For instance, we can think of the call *max*(10, 2 + 3) of the *int max(int x, int y)* function as executing the body of *max* with *x* replaced by 10 and *y* replaced by 5:

10 > 5 ? 10 : 5

This form of pass by value is usually the only parameter-passing mechanism in functional languages.

Pass by Value

Pass by value is also the default mechanism in C++ and Pascal, and is essentially the only parameter-passing mechanism in C and Java.

However, in these languages a slightly different interpretation of pass by value is used: The parameters are viewed as local variables of the procedure, with initial values given by the values of the arguments in the call.

Note that pass by value does not imply that changes cannot occur outside the procedure through the use of parameters. If the parameter has a pointer or reference type, then the value is an address and can be used to change memory outside the procedure.

Pass by Value

For example, the following C function definitely changes the value of the integer pointed to by the parameter *p*:

```
void init_p (int* p)  
{ *p = 0; }
```

On the other hand, directly assigning to *p* does not change the argument outside the procedure:

```
void init_ptr (int* p)  
{ p = (int*) malloc(sizeof(int));
```


Pass by Value

In addition, in some languages certain values are implicitly pointers or references. For example, in C, arrays are implicitly pointers (to the first location of the array).

This means an array value parameter can always be used to change the values stored in the array:

```
void init_p_0 (int p[ ])  
{ p[0] = 0; }
```

Pass by Reference

In pass by reference, an argument is something like a variable with an allocated address. Instead of passing the value of the variable, pass by reference passes the location of the variable, so that the parameter becomes an *alias* for the argument, and any changes made to the parameter occur to the argument as well.

In C++ and Pascal, pass by reference can be specified by using an ampersand (&) after the data type in C++, and a *var* keyword before the variable name in Pascal:

Pass by Reference

```
void inc(int& x) // C++  
{ x++ ; }  
  
procedure inc(var x: integer); (* Pascal *)  
begin  
    x := x + 1;  
end;
```

After a call to *inc(a)* the value of *a* has increased by 1, so that a side effect has occurred. Multiple aliasing is also possible.

Pass by Reference

```
int a;  
  
void yuck (int& x, int& y)  
{ x = 2;  
  y = 3;  
  a = 4;  
}  
...  
yuck (a, a);
```

Inside procedure *yuck* after the call, the identifiers *x* , *y* , and *a* all refer to the same variable, namely, the variable *a*.

Pass by Reference

C can achieve pass by reference by passing a reference or location explicitly as a pointer. C uses the operator `&` to indicate the location of a variable and the operator `*` to *dereference* a pointer. For example:

```
void inc (int* x) /*C imitation of pass by reference */
{ (*x)++; /* adds 1 to *x */ }
...
int a;
...
inc(&a); /* pass the address of a to the inc function */
```

Pass by Reference

One additional issue that must be resolved in a language with pass by reference is the response of the language to reference arguments that are not variables and do not have known addresses within the runtime environment. For example, given the C++ code:

```
void inc(int & x)
{ x++; }
...
inc(2); // ??
```

How should the compiler respond to the call *inc(2)*?

The response of the compiler is to create a temporary integer location, initialize it with the value 2, and then apply the inc function. This actually mimics pass by value, since the change to the temporary location does not affect the rest of the program.

Pass by Value-Result

Pass by value-result achieves a similar result to pass by reference, except that no actual alias is established.

Instead, in pass by value-result, the value of the argument is copied and used in the procedure, and then the final value of the parameter is copied back out to the location of the argument when the procedure exits.

Thus, this method is sometimes known as *copy-in*, *copy-out*, or *copy-restore*.

Pass by Value-Result

Pass by value-result is only distinguishable from pass by reference in the presence of aliasing. Thus, in the following code written in C syntax:

```
void p(int x, int y)
{ x++;
  y++;
}

main()
{ int a = 1;
  p(a, a);
  ...
}
```

a has value 3 after *p* is called if pass by reference is used, while *a* has the value 2 if pass by value-result is used.

Pass by Name and Delayed Evaluation

Pass by name is the term used for this mechanism when it was introduced in Algol60.

The idea of pass by name is that the argument is not evaluated until its actual use as a parameter in the called procedure. Thus, the name of the argument, or its textual representation at the point of call, replaces the name of the parameter to which it corresponds.

Pass by Name and Delayed Evaluation

As an example, in the C code:

```
void inc(int x)  
{ x++; }
```

if a call such as *inc(a[i])* is made, the effect is of evaluating *a[i]++* . Thus, if *i* were to change before the use of *x* inside *inc*.

Pass by Name and Delayed Evaluation

Pass by name can be interpreted as follows:

The text of an argument at the point of call is viewed as a function in its own right, which is evaluated every time the corresponding parameter name is reached in the code of the called procedure.

However, the argument will always be evaluated in the environment of the caller, while the procedure will be executed in its defining environment.

To see how this works, consider the next example:

Pass by Name and Delayed Evaluation

To see how this works, consider the the example below:

```
(1)  #include <stdio.h>
(2)  int i;

(3)  int p(int y)
(4)  { int j = y;
(5)    i++;
(6)    return j+y;
(7)  }

(8)  void q(void)
(9)  { int j = 2;
(10)   i = 0;
(11)   printf("%d\n", p(i + j));
(12) }

(13) main()
(14) { q();
(15)   return 0;
(16) }
```

The argument $i + j$ to the call to p from q is evaluated every time the parameter y is encountered inside p . The expression $i + j$ is, however, evaluated as though it were still inside q , so on line 4 in p it produces the value 2. Then, on line 6, since i is now 1, it produces the value 3 (the j in the expression $i + j$ is the j of q , so it hasn't changed, even though the i inside p has). Thus, if pass by name is used for the parameter y of p in the program, the program will print 5.

Type Checking of Parameters

In strongly typed languages, procedure calls must be checked so that the arguments agree in type and number with the parameters of the procedure.

This means, that procedures may not have a variable number of parameters and that rules must be stated for the type compatibility between parameters and arguments.

In the case of pass by reference, parameters usually must have the same type, but in the case of pass by value, this can be relaxed to assignment compatibility, and can allow conversions, as is done in C, C++, and Java. Ada, of course, does not allow conversions.

Assessment Exercises



To assess the students understanding of the topic and continue the discussion, please answer the following exercise questions:

1. Beginning programmers often add a return statement at the end of each procedure:

```
void inc (int* x)
{
    (*x)++;
    return;
}
```

The thinking is that, as with switch statements, there might be a “fall-through” if the return is not there. Could a function mechanism be designed that would allow this? Would it make sense to have such a mechanism?

Assessment Exercises



To assess the students understanding of the topic and continue the discussion, please answer the following exercise questions:

2. What if we wrote an increment procedure in C as follows (removing the parentheses around `*x`):

```
void inc (int* x)
{  *x++;
}
```

Is this legal? If so, what does this procedure do? If not, why not?